

Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAXM 100005
#define MAXN 5005

typedef struct {
    int u, v, w;
} Edge;

Edge heap[MAXM];
int heap_size = 0;

void heapify(int i) {
    int largest = i;
    int left = 2 * i;
    int right = 2 * i + 1;

    if (left <= heap_size && heap[left].w > heap[largest].w)
        largest = left;
    if (right <= heap_size && heap[right].w > heap[largest].w)
        largest = right;

    if (largest != i) {
        Edge temp = heap[i];
        heap[i] = heap[largest];
        heap[largest] = temp;
        heapify(largest);
    }
}

void insert_edge(Edge e) {
    heap_size++;
    int i = heap_size;
    heap[i] = e;
    while (i > 1 && heap[i / 2].w < heap[i].w) {
        Edge temp = heap[i];
        heap[i] = heap[i / 2];
        heap[i / 2] = temp;
        i = i / 2;
    }
}
```

```

    }
}

Edge extract_max() {
    Edge max_edge = heap[1];
    heap[1] = heap[heap_size];
    heap_size--;
    heapify(1);
    return max_edge;
}

int parent[MAXN];

int find(int i) {
    if (parent[i] == i) return i;
    return parent[i] = find(parent[i]);
}

int unite(int i, int j) {
    int root_i = find(i);
    int root_j = find(j);
    if (root_i != root_j) {
        parent[root_i] = root_j;
        return 1;
    }
    return 0;
}

int printheap(int N) {
    for (int i = 1; i <= N; i++) parent[i] = i;
    int edges_used = 0;
    long long total_weight = 0;

    while (heap_size > 0 && edges_used < N - 1) {
        Edge e = extract_max();
        if (unite(e.u, e.v)) {
            total_weight += e.w;
            edges_used++;
        }
    }
    return (int)total_weight;
}

int main() {

```

```
int T;
if (scanf("%d", &T) != 1) return 0;

while (T--) {
    int N, M;
    scanf("%d %d", &N, &M);
    heap_size = 0;

    for (int i = 0; i < M; i++) {
        Edge e;
        scanf("%d %d %d", &e.u, &e.v, &e.w);
        insert_edge(e);
    }

    printf("%d\n", printheap(N));
}
return 0;
}
```

Input:

```
1
3 3
1 2 2
2 3 3
1 3 4
```

Output:

```
7
```